Programação Orientada a Objetos

Prof. Henrique Batista da Silva

Programação Orientada a Objetos

"**Programação orientada a objetos**: nasceu porque algumas **linguagens procedimentais** se mostraram inadequadas para a construção de programas de grande porte." Nivio Ziviane

Programação Orientada a Objetos

- Conforme apresentado anteriormente, em uma linguagem orientada a objetos como o Java, podemos representar qualquer entidade do mundo real por meio de **classes**.

- A classe nos permite definir uma entidade do mundo real, seus **atributos** (variáveis) que a caracterizam e **métodos** (funções) para acessar estes atributos.

Programação Orientada a Objetos

- Um empregado, por exemplo, por ser definido em um sistema por meio de uma classe **Empregado**. Esta classe conterá os atributos de empregado, como ***matrícula***, ***nome***, e ***salário base***.
- Ela também pode ter um método que calcula o salario total a partir da quantidade de horas extras realizadas.

```
class Empregado {
        int matricula;
        String nome;
        double salarioBase;

        double calculaSalario (double horaExtra) {
                ... Código do método
        }
}
```

Programação Orientada a Objetos

- Ao criar uma nova classe, como a classe Empregado do exemplo anterior, estamos definindo um novo tipo de dado.

- Os tipos de dados como **int**, **double** e **String**, utilizados até então, são tipos primitivos da linguagem. Por meio destes tipos podemos criar novas ***variáveis***.

- Usamos classes para criar tipos não primitivos e, a partir de sua definição, podemos criar **objetos**, que são instâncias de uma classe. Ou seja, o **objeto** é uma variável do tipo da classe.

Programação Orientada a Objetos

- Assim, a **Programação Orientada a Objetos (OO)** permite que objetos do mundo real que compartilham propriedades e comportamentos comuns sejam agrupados em classes.

- Estilo de programação diretamente suportado pelo conceito de classe em Java.

- Ela permite melhorar a **reusabilidade** e **extensibilidade** dos softwares.

Mecanismo de Reuso de código

- Diminui a necessidade de reescrever código:
  - menos trabalho para o programador.
- Permite o aproveitamento de código preexistente (livre de erro e otimizado):
  - menos chances de cometer erros
- Em linguagens convencionais:
  - biblioteca de funções e procedimentos

Mecanismo de Reuso de código

- Biblioteca **Random** do Java para gerar números aleatórios.
- Objeto chamado ***rand*** do tipo **Random**.
- Chamada do método ***nextInt()*** para retornar um número inteiro aleatório.

```
import java.util.Random;

public class Principal {
   public static void main(String[] args) {
      Random rand = new Random();

      for (int i = 0; i < 5; i++) {
         int aleatorio = rand.nextInt() % 10;
         System.out.println(aleatorio + " ");
      }
   }
}
```

Mecanismo de Reuso de código

- Em nenhum momento foi necessário implementar um novo código para gerar números aleatórios.

- Foi feito reuso de um método pronto de uma biblioteca fornecida pela linguagem, por meio da criação de um novo objeto.

```
import java.util.Random;

public class Principal {
   public static void main(String[] args) {
         Random rand = new Random();

         for (int i = 0; i < 5; i++) {
            int aleatorio = rand.nextInt() % 10;
            System.out.println(aleatorio + " ");
         }
   }
}
```

Conceitos básicos de OO

- Em OO os objetos da computação são análogos aos objetos existentes no mundo real.

- Os átomos do processo de computação são os objetos que trocam **mensagens** entre si.

- Essas mensagens resultam na ativação de **métodos**, os quais realizam as ações necessárias

Conceitos básicos de OO

- Objeto é algo **dinâmico**: criado por alguém, tem uma vida, morre ou é morto por alguém. Assim durante a execução do sistema, os objetos podem: ser construídos, executar ações, ser destruídos, tornar-se inacessíveis.

- A classe é apenas uma definição (especificação). O objeto é uma instância da classe, quando a mesma ganha vida. Quando um objeto fica inacessível, por exemplo, ele deixa de existir. Porém, outros objetos podem ser criados com a mesma especificação.

Objeto

- É uma entidade capaz de reter um **estado** (informação) e que oferece uma série de **operações** (comportamento) ou para examinar ou para afetar este estado.

- Eles pode ser:
  - Concretos:
    - Carro, casa, avião
  - Abstratos:
    - Transação bancária, música

Exemplo Cachorro

- Possui várias características
  - Um nome,
  - Uma idade,
  - Um comprimento de pelos,
  - Uma cor dos pelos,
  - Uma cor dos olhos,
  - Um peso,
  - ...

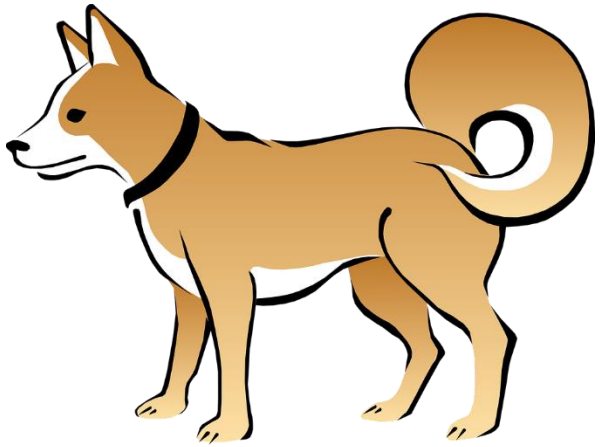


Imagem: cdn.pixabay.com

As características que descrevem um objeto são chamadas de **atributos**

Atributo

- Os objetos do mundo real possuem propriedades que possuem valores.
- Estes valores definem o **estado** do objeto. As propriedades recebem o nome de atributos em OO.
- Elementos que caracterizam o objeto.
- Cada objeto tem seu conjunto próprio de atributos.
- **Variável declarada no contexto da classe.**

Exemplo Cachorro

Cachorro	
Nome	Rex
Idade	2 anos
Comprimento dos pelos	Curtos
Cor dos pelos	Bege
Cor dos olhos	Castanhos
Peso	6 Kg

Imagem: upload.wikimedia.org

Cachorro	
Nome	Hachiko
Idade	5 anos
Comprimento dos pelos	Curtos
Cor dos pelos	Bege
Cor dos olhos	Castanhos
Peso	8 Kg

Imagem: upload.wikimedia.org

Exemplo Cachorro

- Além do conjunto de características que descrevem o cachorro, podemos também identificar um conjunto de ações que ele é capaz de executar:
  - Latir,
  - Babar,
  - Correr em círculos,
  - Pegar bola,
  - Sentar,
  - Comer,
  - Dormir,
  - ...

Chamamos de interface ao conjunto de métodos disponíveis em um objeto

Método

- São comportamentos inerentes ao objeto, executados somente quando são solicitados.

- São procedimentos ou funções que realizam as ações próprias do objeto.

- Um objeto requisita a ação de algum outro objeto, enviando uma **mensagem** para ele.

Método

- Novamente vamos rever o objeto ***rand*** criado da classe **Random** para gerar números aleatórios.

- Estamos requisitando uma ação fazendo a chamada do método ***nextInt()*** do objeto ***rand*** para retornar um número inteiro aleatório.

```
import java.util.Random;

public class Principal {
   public static void main(String[] args) {
         Random rand = new Random();

         for (int i = 0; i < 5; i++) {
            int aleatorio = rand.nextInt() % 10;
            System.out.println(aleatorio + " ");
         }
   }
}
```

Classe

- Uma **classe** representa um conjunto de objetos que possuem características e comportamentos

- Um **objeto** é uma instância de uma determinada **classe**, ou seja, criamos nossos objetos baseados nas características definidas nas classes.

- A ênfase da metodologia orientada a objetos é dada na criação das classes, e não dos objetos, como se poderia pensar pelo nome.

Exemplo de uma classe

```
class Bicicleta {

        private String modelo;
        private double preco;
        private int velocidade;

        public void Acelerar (int kmph) {
           velocidade = velocidade + kmph;
        }

        public void frear () {
           if (velocidade > 0)
              velocidade = velocidade - 1;
        }

        public int consultarVelocidade () {
           return velocidade;
        }
}
```

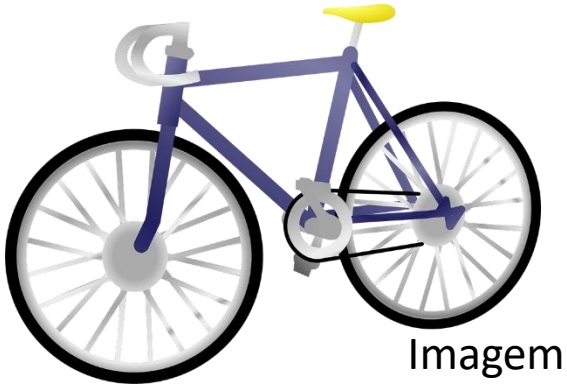


Imagem: cdn.pixabay.com

```
Bicicleta bike = new Bicicleta();
```

Instanciação da classe Bicicleta

Encapsulamento

- O encapsulamento é o resultado (ou ato) de ocultar do usuário os detalhes da implementação de um objeto.
- O encapsulamento é importante porque separa a maneira como um objeto se comporta da maneira como ele é implementado.
- A definição de como implementar as ações de uma classe**, sem informar como isto é feito.**

```
public static void main(String[] args) {
        Bicicleta bike = new Bicicleta();

        bike.Acelerar(12);
}
```

Ao criar um objeto da classe bicicleta do exemplo anterior, o encapsulamento não permite acesso à implementação dos métodos. Não sabemos como ele é feito, e apenas o utilizamos.

Encapsulamento

- Outras classes só podem acessar os atributos de uma classe invocando os métodos públicos;
- Restringe a visibilidade do objeto, mas facilita o reuso

```
public static void main(String[] args) {
        Bicicleta bike = new Bicicleta();

        bike.Acelerar(12);
}
```

Ao criar um objeto da classe bicicleta do exemplo anterior, o encapsulamento **não** permite que o atributo ***velocidade*** da bicicleta seja acessado (e modificado) de "fora" da classe.

Classe conta bancária

- Exemplo simples: Conta Bancaria
  - Queremos armazenar os dados da conta.
  - Queremos ser capazes de efetuar operações com os
    - dados da conta (alguns podem ser modificados, outros não, alguns ainda somente sob certas condições)...

Classe conta bancária

```
class ContaBancaria {
        int codCorrentista;
        double saldo;

        ContaBancaria (int cod, double s) {
                codCorrentista = cod;
                saldo = s;
        }

        double getSaldo () { return saldo; }

        int getCod() { return codCorrentista; }

        void deposita (double quantia) { saldo += quantia; }

        void retira (double quantia) {
                if (quantia < saldo) saldo -= quantia;
        }
}
```

Classe conta bancária

```
class ContaBancaria {
        int codCorrentista;
        double saldo;

        ContaBancaria (int cod, double s) {
                codCorrentista = cod;
                saldo = s;
        }

        double getSaldo () { return saldo; }

        int getCod() { return codCorrentista; }

        void deposita (double quantia) { saldo += quantia; }

        void retira (double quantia) {
                if (quantia < saldo) saldo -= quantia;
        }
}
```

Nome da classe. Normalmente sempre declaramos nome de classe começando com letra maiúscula

Classe conta bancária

```
class ContaBancaria {
        int codCorrentista;
        double saldo;

        ContaBancaria (int cod, double s) {
                codCorrentista = cod;
                saldo = s;
        }

        double getSaldo () { return saldo; }

        int getCod() { return codCorrentista; }

        void deposita (double quantia) { saldo += quantia; }

        void retira (double quantia) {
                if (quantia < saldo) saldo -= quantia;
        }
}
```

Atributos da classe ContaBancaria.

Classe conta bancária

```
class ContaBancaria {
        int codCorrentista;
        double saldo;

        ContaBancaria (int cod, double s) {
                codCorrentista = cod;
                saldo = s;
        }

        double getSaldo () { return saldo; }

        int getCod() { return codCorrentista; }

        void deposita (double quantia) { saldo += quantia; }

        void retira (double quantia) {
                if (quantia < saldo) saldo -= quantia;
        }
}
```

Construtor da classe. É uma método especial que não contém tipo (ele é do tipo da classe) e, consequentemente, não retorna nada.

Ele inicializa os atributos da classe. Será chamado sempre que instanciar um novo objeto. Por exemplo:

```
ContaBancaria richard = new ContaBancaria
(1, 2000);
```

Observe que o construtor recebe dois parâmetros, um do tipo **String** e do tipo **double**. É necessário informar os valores deste parâmetros que são fornecidos para a classe.

Classe conta bancária

```
class ContaBancaria {
        int codCorrentista;
        double saldo;

        ContaBancaria (int cod, double s) {
                codCorrentista = cod;
                saldo = s;
        }

        double getSaldo () { return saldo; }

        int getCod() { return codCorrentista; }

        void deposita (double quantia) { saldo += quantia; }

        void retira (double quantia) {
                if (quantia < saldo) saldo -= quantia;
        }
}
```

Métodos da classe ContaBancaria. São funções ou procedimentos que podem ser chamados pelo objeto (conforme veremos a seguir).

Classe conta bancária

```
Class UsaContaBancaria {
   public static void main (String[] args) {
        ContaBancaria fred = new ContaBancaria (1, 1000);
        ContaBancaria richard = new ContaBancaria (2, 2000);
        richard.retira(500);
        fred.retira(500);
   }
}
```

Dois objetos são criados do tipo da classe ContaBancaria. O operador **new** é usado para instanciar os objetos. Neste momento, o construtor da classe será acionado.

Classe conta bancária

```
Class UsaContaBancaria {
   public static void main (String[] args) {
         ContaBancaria fred = new ContaBancaria (1, 1000);
         ContaBancaria richard = new ContaBancaria (2, 2000);
         richard.retira(500);
         fred.retira(500);
   }
}

Class UsaContaBancariaComErros {
   public static void main (String[] args) {
         ContaBancaria fred = new ContaBancaria (1);
         ContaBancaria richard = new ContaBancaria (2000, 2);
         richard.retira();
         fred.desfalca();
         rechard.saldo = 10000;
   }
}
```

Falta parâmetro

Ordem incorreta

Falta especificar a quantidade

Método da classe inexistente

Isto não é um erro!

Classe conta bancária (pequena alteração)

```
class ContaBancaria {
        private int codCorrentista;
        private double saldo;

        public ContaBancaria (int cod, double s) {
                codCorrentista = cod;
                saldo = s;
        }

        public double getSaldo () { return saldo; }

        public int getCodCorrentista() { return codCorrentista; }

        public void deposita (double quantia) { saldo += quantia; }

        public void retira (double quantia) {
                if (quantia < saldo) saldo -= quantia;
        }
}
```

Classe conta bancária (pequena alteração)

```
class ContaBancaria {
        private int codCorrentista;
        private double saldo;

        public ContaBancaria (int cod, double s) {
                codCorrentista = cod;
                saldo = s;
        }

        public double getSaldo () { return saldo; }

        public int getCodCorrentista() { return codCorrentista; }

        public void deposita (double quantia) { saldo += quantia; }

        public void retira (double quantia) {
                if (quantia < saldo) saldo -= quantia;
        }
}
```

Private: O membro **private** não é acessível fora da classe. A intenção aqui é permitir que apenas você que escreve a classe possa usar esse membro

Classe conta bancária (pequena alteração)

```
class ContaBancaria {
        private int codCorrentista;
        private double saldo;

        public ContaBancaria (int cod, double s) {
                codCorrentista = cod;
                saldo = s;
        }

        public double getSaldo () { return saldo; }

        public int getCodCorrentista() { return codCorrentista; }

        public void deposita (double quantia) { saldo += quantia; }

        public void retira (double quantia) {
                if (quantia < saldo) saldo -= quantia;
        }
}
```

Public: Quem tem acesso à classe tem acesso também a qualquer membro com visibilidade **public**.
O alvo aqui é o programador cliente que usa suas classes. É raro ter atributos públicos mas é comum ter métodos públicos

Classe conta bancária (pequena alteração)

```
Class UsaContaBancaria {
   public static void main (String[] args) {
        ContaBancaria fred = new ContaBancaria (1, 10);
        ContaBancaria richard = new ContaBancaria (2, 20);
        richard.retira(5);
        richard.saldo = 1000000;
   }
}
```

Erro de compilação!

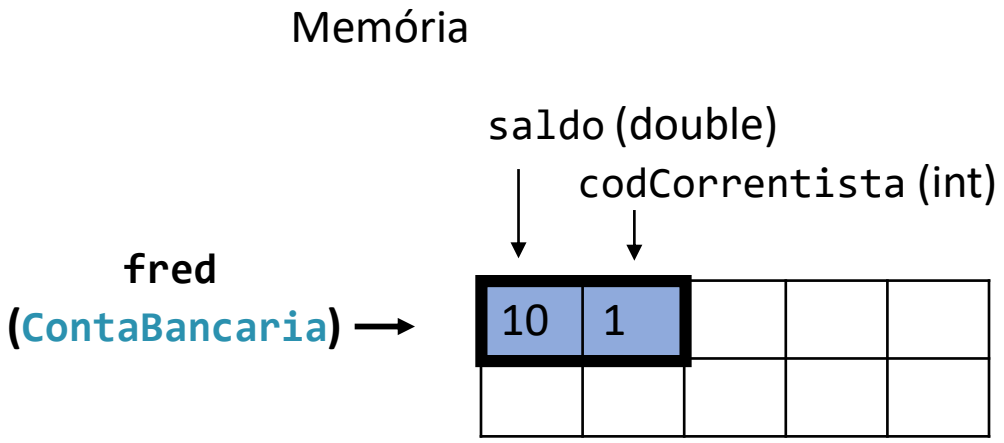
O atributo saldo, após a nossa modificação, é **privado**. Assim, ele não pode ser alterado diretamente como no exemplo acima. Apenas os métodos da classe, como o método “retira ()” e “deposita ()” podem alterar o atributo saldo.

Classe conta bancária (pequena alteração)

```
Class UsaContaBancaria {
   public static void main (String[] args) {
        ContaBancaria fred = new ContaBancaria (1, 10);
        ContaBancaria richard = new ContaBancaria (2, 20);
        richard.retira(5);
        fred.deposita(5);
    }
}
```

Uma classe é uma especificação de um tipo de dado. Nós não manipulamos a classe em si, e sim os objetos que criamos a partir da classe.

Encapsulamento

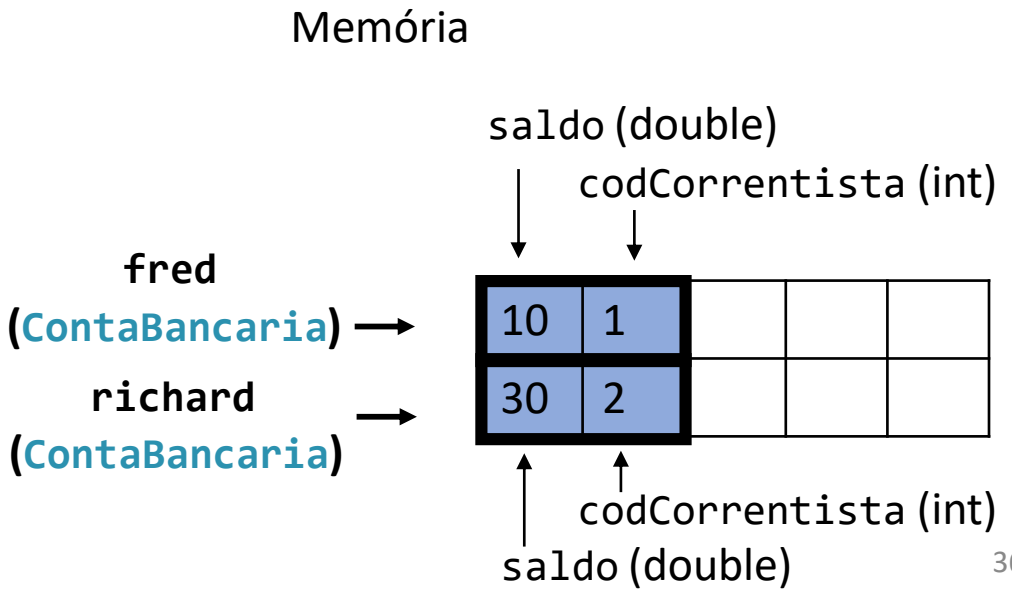


Classe conta bancária (pequena alteração)

```
Class UsaContaBancaria {
   public static void main (String[] args) {
        ContaBancaria fred = new ContaBancaria (1, 10);
        ContaBancaria richard = new ContaBancaria (2, 20);
        richard.retira(5);
        fred.deposita(5);
    }
}
```

Uma classe é uma especificação de um tipo de dado. Nós não manipulamos a classe em si, e sim os objetos que criamos a partir da classe.

Encapsulamento

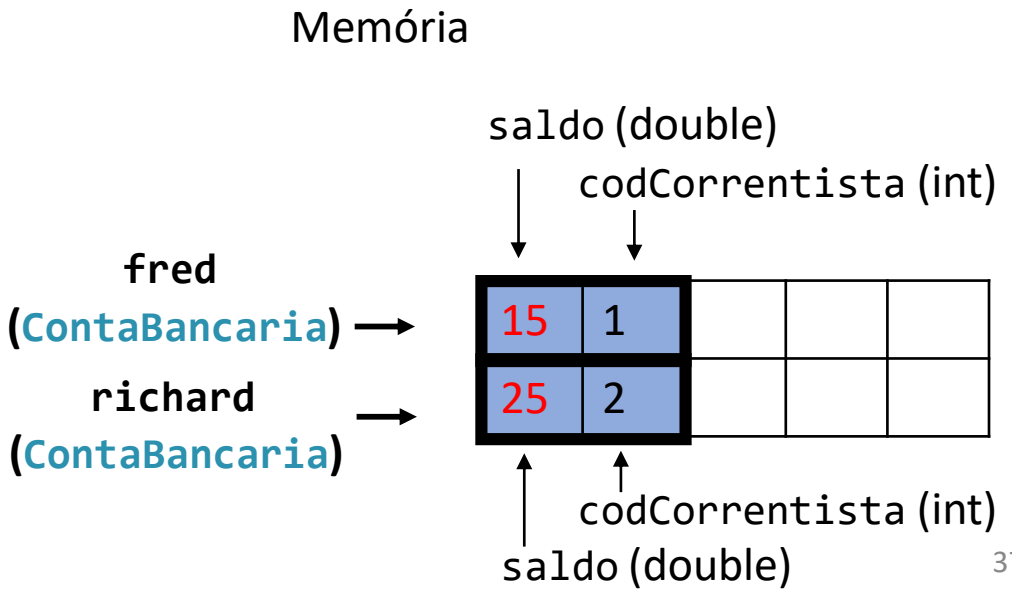


Classe conta bancária (pequena alteração)

```
Class UsaContaBancaria {
   public static void main (String[] args) {
        ContaBancaria fred = new ContaBancaria (1, 10);
        ContaBancaria richard = new ContaBancaria (2, 20);
        richard.retira(5);
        fred.deposita(5);
    }
}
```

Uma classe é uma especificação de um tipo de dado. Nós não manipulamos a classe em si, e sim os objetos que criamos a partir da classe.

Para manipular os atributos dos objetos **richard** e **fred**, é necessário invocar os métodos dos respectivos objetos. Somente por eles é possível alterar os seus atributos.



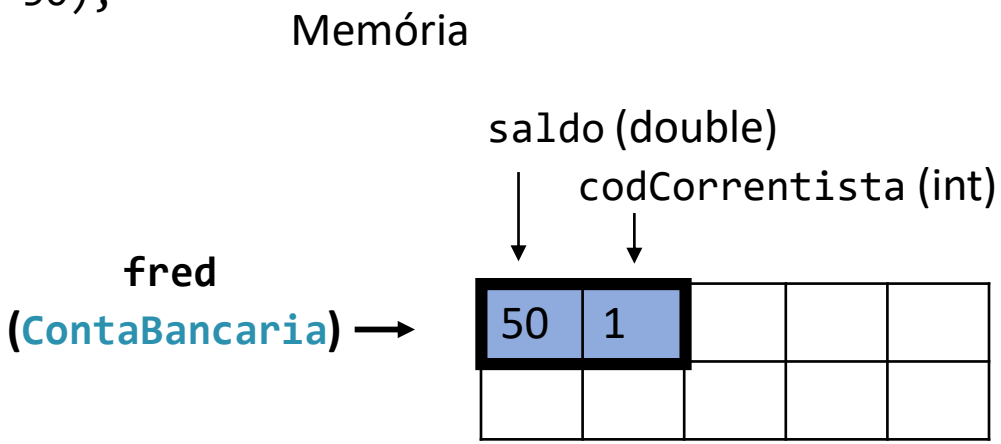
Passagem de objetos por referência

- Como ocorre com os vetores, os objetos de classes também são passados por parâmetros para as funções por referência.

- Ao passar um objeto de uma classe para uma função, se a função realizar alterações no objeto, estas alterações serão refletidas ao objeto quando a função foi chamada.

Passagem de objetos por referência

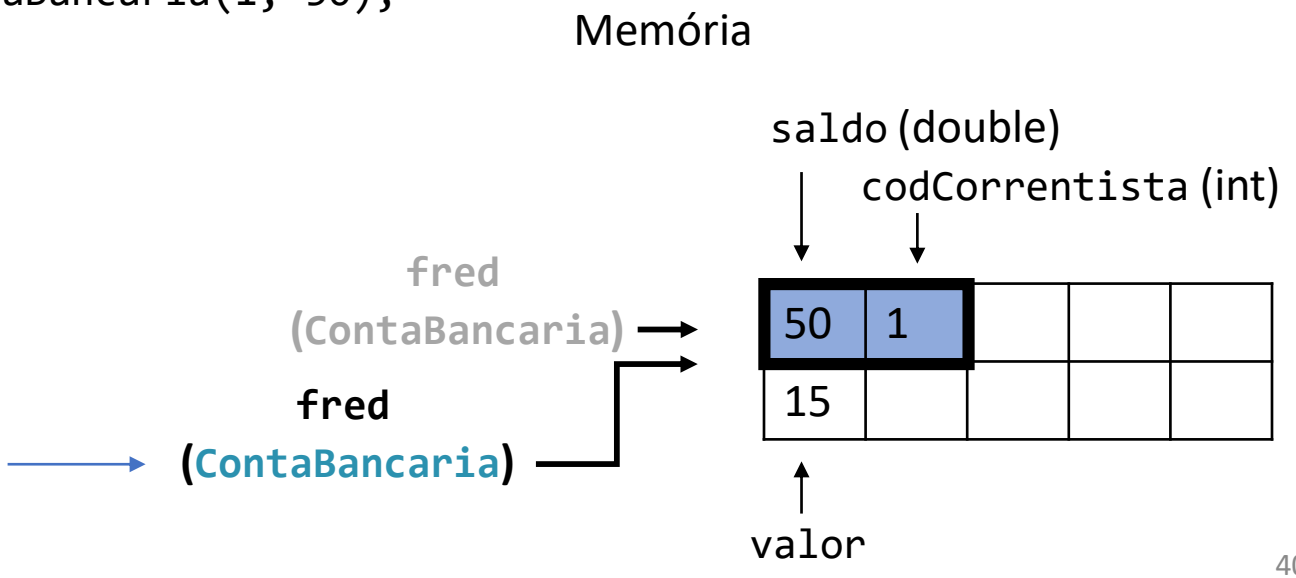
```
Class UsaContaBancaria {

    public static void pagaBoleto (ContaBancaria fred, double valor)
    {
        fred.retira(valor);
    }

    public static void main (String[] args) {
        ContaBancaria fred = new ContaBancaria(1, 50);
        pagaBoleto (fred, 15);
    }
}
```



Passagem de objetos por referência

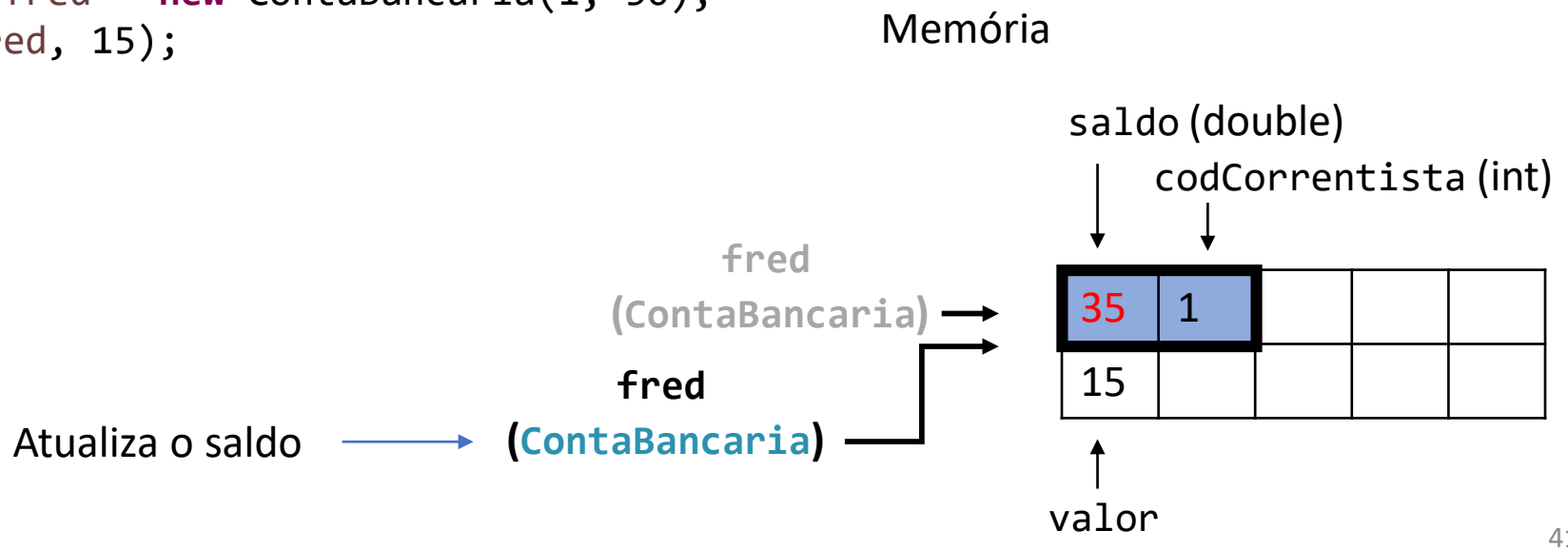
```
Class UsaContaBancaria {

    public static void pagaBoleto (ContaBancaria fred, double valor)
    {
        fred.retira(valor);
    }

    public static void main (String[] args) {
        ContaBancaria fred = new ContaBancaria(1, 50);
        pagaBoleto (fred, 15);
    }
}
```

Um novo objeto é criado e recebe como parâmetro o mesmo endereço de memória do objeto da função principal.

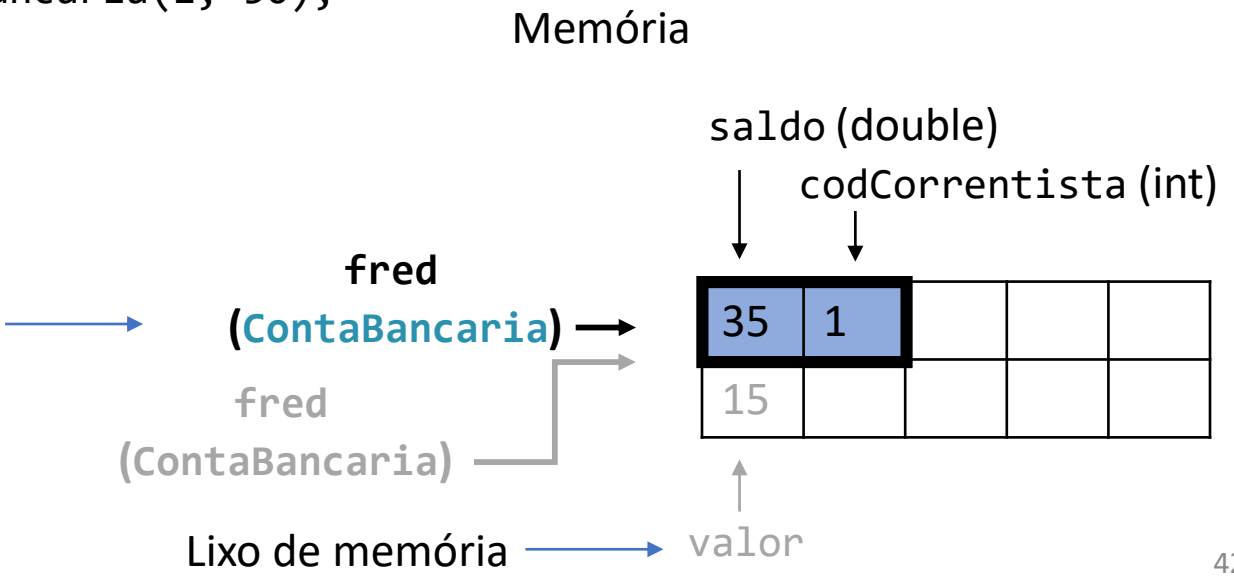


Passagem de objetos por referência

```
Class UsaContaBancaria {

    public static void pagaBoleto (ContaBancaria fred, double valor)
    {
        fred.retira(valor);
    }

    public static void main (String[] args) {
        ContaBancaria fred = new ContaBancaria(1, 50);
        pagaBoleto (fred, 15);
    }
}
```



Passagem de objetos por referência

```
Class UsaContaBancaria {

    public static void pagaBoleto (ContaBancaria fred, double valor)
    {
        fred.retira(valor);
    }

    public static void main (String[] args) {
        ContaBancaria fred = new ContaBancaria(1, 50);
        pagaBoleto (fred, 15);
    }
}
```

Observe que as alterações realizadas dentro da função são refletidas no objeto original